

Python Core & OOP Reference Guide

Functions, Variable Scopes (LEGB), Exception Handling, and Object-Oriented Programming Patterns

1. Python Functions: Fundamentals & Anatomy

A **function** is a reusable, named block of statements designed to perform a specific operation. Functions promote modularity, prevent duplication (DRY principle), and serve as first-class objects in Python.

```
# Syntax Anatomy of a Python Function
def calculate_tax(gross_income, tax_rate=0.15, *exemptions, **deductions):
    """Calculates net payable income tax based on rate and bracket deductions."""
    taxable = gross_income - sum(exemptions)
    total_deductions = sum(deductions.values())
    net_tax = (taxable * tax_rate) - total_deductions
    return max(0.0, net_tax)
```

Parameter Types

- **Positional:** Bound strictly by position/order.
- **Keyword:** Bound by explicit argument name.
- **Default:** Holds fallback value if omitted.
- ***args:** Gathers excess positional arguments into a `tuple`.
- ****kwargs:** Gathers extra keyword arguments into a `dict`.

Evaluation & Pass-by-Assignment

Python utilizes **Pass-by-Assignment** (Pass-by-Object-Reference):

- **Immutable objects** (`int`, `str`, `tuple`): Cannot be altered inside the function body.
- **Mutable objects** (`list`, `dict`, `set`): Modifications in-place reflect directly in the caller's scope.

Critical Edge Case: Mutable Default Arguments

Default parameter values are evaluated *once* at function definition time. Avoid `def func(lst=[])`. Instead, use `None` as the default marker and assign dynamically within the function block.

```
# Safe Idiomatic Handling for Mutable Defaults
def append_transaction(val, tracker=None):
    if tracker is None:
        tracker = [] # Fresh list instantiated on each invocation
    tracker.append(val)
    return tracker
```

2. Variable Scope & The LEGB Lookup Rule

Scope determines the exact region of code where a variable name can be recognized and evaluated. Python resolves identifier names using the strict **LEGB Rule Order**.

L - Local Scope (Inside current function block)

E - Enclosing Scope (Outer/Nested functions)

G - Global Scope (Top-level module file)

B - Built-in Scope (Preloaded Python library built-ins)

Rebinding Names with `global` and `nonlocal`

By default, assignment within a function creates a new local variable. Modifying variables from outer scopes requires explicit scope modifiers.

```
# Scope Binding Example
system_status = "OFFLINE" # Global Scope

def outer_controller():
    process_count = 0 # Enclosing Scope

    def inner_worker():
        global system_status
        nonlocal process_count

        system_status = "ONLINE" # Rebinds Global Variable
        process_count += 1 # Rebinds Enclosing Variable

    inner_worker()
    print(f"Active Processes: {process_count}") # Output: 1

outer_controller()
print(f"System State: {system_status}") # Output: ONLINE
```

Scope	Declaration Point	Lifetime Duration	Modification Command
Local	Inside function body / parameter	During function execution	Standard assignment (=)
Enclosing	Inside outer nested function	Until outer function scope clears	<code>nonlocal var_name</code>
Global	Top level of current module file	Duration of program process	<code>global var_name</code>
Built-in	Pre-defined Python module space	Interpreter process lifetime	Avoid overriding built-ins

3. Robust Error & Exception Handling

Runtime errors produce **Exceptions** that break script execution if unhandled. Python provides a structured framework (`try-except-else-finally`) to handle potential failures gracefully.

```
# Complete Exception Control Logic
def execute_transaction(account_id, amount):
    try:
        # Risky Block
        account = fetch_account(account_id)
        account.withdraw(amount)
    except FileNotFoundError as err:
        print(f"[Error] Record file missing: {err}")
    except (ValueError, TypeError) as err:
        print(f"[Error] Invalid transaction payload: {err}")
    except Exception as err:
        print(f"[Error] Unexpected fault: {err}")
    else:
        # Executes ONLY if NO exceptions occurred in try block
        print("[Success] Transaction posted cleanly.")
    finally:
        # ALWAYS executes (Used for resource cleanup/closing handles)
        close_database_connection()
```

Custom Exception Hierarchy

Domain-specific applications should inherit from standard Python's base `Exception` class to provide granular error classification.

```
# Domain Custom Exception Definition
class InsufficientBalanceError(Exception):
    """Raised when account balance is lower than request withdrawal."""
    def __init__(self, current, requested):
        self.current = current
        self.requested = requested
        super().__init__(f"Shortfall of ${requested - current}. Balance: ${current}")

class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientBalanceError(self.balance, amount)
        self.balance -= amount
        return self.balance
```

Best Practice Guidelines

- Avoid bare `except:` blocks; catch precise exceptions to prevent intercepting system signals like `KeyboardInterrupt`.
- Keep code inside the `try` block strictly focused on statements that might raise an exception.

4. Object-Oriented Programming (OOP) Essentials

Object-Oriented Programming models software design around **Objects**—data collections bound with behavioral methods. A **Class** serves as the blueprint, while an **Instance** is the concrete object formed from it.

```
# Core OOP Class Structure
class Employee:
    company_name = "TechCorp" # Class Attribute (Shared across all instances)

    def __init__(self, name, salary):
        self.name = name # Instance Attribute (Unique to object)
        self.salary = salary # Instance Attribute

    # 1. Instance Method: Receives instance binding (self)
    def get_details(self):
        return f"Employee: {self.name} | Salary: ${self.salary}"

    # 2. Class Method: Receives class binding (cls)
    @classmethod
    def from_string(cls, data_str):
        name, salary = data_str.split(",")
        return cls(name, float(salary))

    # 3. Static Method: Unbound utility function
    @staticmethod
    def is_valid_salary(val):
        return val >= 30000
```

Instance Methods

Takes `self` as first argument. Used for inspecting or modifying instance specific state attributes.

Class Methods (@classmethod)

Takes `cls` as first argument. Used for factory constructors and class-level attribute alterations.

Property Decorators (Controlled Access)

```
class SmartLight:
    def __init__(self, brightness=100):
        self._brightness = brightness # Protected attribute convention

    @property
    def brightness(self):
        return self._brightness

    @brightness.setter
    def brightness(self, val):
        if not (0 <= val <= 100):
            raise ValueError("Brightness must be between 0 and 100")
        self._brightness = val
```

5. The Four Pillars of Object-Oriented Programming

1. Encapsulation

Restricting direct attribute access using naming conventions:

- `public` : Accessible anywhere.
- `_protected` : Subclass/internal hint.
- `__private` : Triggers name mangling.

2. Abstraction

Hiding background implementation via the `abc` module. Abstract classes require subclasses to implement annotated methods.

```
# Abstraction and Encapsulation Example
from abc import ABC, abstractmethod

class PaymentGateway(ABC):
    @abstractmethod
    def process_payment(self, amount):
        pass # Mandatory implementation target for subclasses

class StripeGateway(PaymentGateway):
    def __init__(self, api_key):
        self.__api_key = api_key # Private Attribute

    def process_payment(self, amount):
        print(f"Charged ${amount} via Stripe Key {self.__api_key[:4]}**")
```

3. Inheritance

Child classes acquire methods and state attributes from parent classes using `super()` calls.

4. Polymorphism

Different classes responding uniquely to the same unified method call signature.

```
# Inheritance and Polymorphism Example
class Vehicle:
    def __init__(self, brand):
        self.brand = brand
    def drive(self):
        return "Vehicle is moving"

class ElectricCar(Vehicle):
    def __init__(self, brand, battery_capacity):
        super().__init__(brand)
        self.battery_capacity = battery_capacity
    def drive(self):
        return f"Silent electric glide in {self.brand}" # Method Override

fleet = [Vehicle("Ford"), ElectricCar("Tesla", 85)]
for v in fleet:
    print(v.drive()) # Polymorphic invocation
```

6. Advanced Dunder Methods & Exercises

Dunder (Double Underscore) Methods customize Python operator behavior and core language interactions for custom objects.

```
# Magic / Dunder Method Implementation
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self): # Human readable string format (print/str)
        return f"Vector({self.x}, {self.y})"

    def __add__(self, other): # Overloads '+' operator
        return Vector(self.x + other.x, self.y + other.y)

    def __eq__(self, other): # Overloads '==' equality check
        return self.x == other.x and self.y == other.y

v1 = Vector(2, 4)
v2 = Vector(3, 1)
print(v1 + v2) # Output: Vector(5, 5)
```

7. Self-Assessment Practice Exercises

Exercise 1: Closures and State

Build a function `make_accumulator(start_value)` that returns an inner function capable of maintaining an ongoing sum across calls without global variables.

Exercise 2: Custom Exception Handler

Construct a custom exception `InvalidAgeError`. Write a function `register_voter(age)` that raises this error if `age < 18`, handling it with standard try-except blocks.

Exercise 3: Polymorphic Banking System

Design an abstract class `Account` with a method `calculate_interest()`. Create subclasses `SavingsAccount` and `CheckingAccount` that yield distinct interest returns.